

Exercise: Spatial Position and Adjustment

Overview

This exercises provides some practice manipulating the spatial position of points and bars in visualizations and is designed to be completed in 60 minutes or fewer. Although you should have time to complete the parts, if you are unable, *as with all exercises, you are encouraged to complete it outside of class time* so that you are able to incorporate your experiences and knowledge into future exercises. You may need to consult course reading materials located at the course site as some elements may not have been covered in the basic content contained in associated videos.

This exercise assumes you have an understanding of `{dplyr}` functions like `select()`, `filter()`, `mutate()`, and `summarize()` and `{ggplot2}` functions like `ggplot()`, `geom_point()`, `geom_bar()`, and as `aes()`. Your ability to work through the exercise will also be influenced in part by your prior practice.

This exercise focuses on:

- Intentional File Creation and File Management
- Creating point plots and bar plots
- Mapping variables to aesthetics
- Adjusting spatial position of geoms
- Combining plots into Grobs

This exercise uses:

- Your RStudio Git version-control project
- `{here}`, `{dplyr}`, `{ggplot2}` functions and functions from relevant Base R libraries (e.g., `read.csv()`, `readRDS()`, `saveRDS()`, etc.)
- Your knowledge from past in-class exercises, videos, homework, etc. and corresponding modules from the course site.

Recommendation: Use team project data inside your `dataviz-exercises` project for now.

Part 1: Creating a Point Plot: Default Overplotting

Using a data set, identify some relevant numeric and non-numeric variables for creating a point plot with a meaningful numeric variable along the y-axis. Either a numeric or non-numeric variable can be used along the x-axis. Consider variables of any type to map to an aesthetic other than x or y.

1. Create an .R script file (Recommendation: Start with `starter_script_file.R`)
2. Name the file appropriately based on the goal of the three parts (review briefly). Add either your name or initials as a prefix to the file name. Do not use spaces.
3. In a “library” section of your script, load your libraries.
4. In a “data-reading” section of your script, write code to read your data. Assign the data frame an object name that unambiguously represents the data you are reading. If you are not using `{here}`, make sure you do.
5. Write code to pipe the data frame to `{ggplot2}` functions in order to create a simple point plot using `geom_point()`. Map a numeric variable to an aesthetic other than x or y. Set an aesthetic as well so that you have practice doing so.
6. Whether there are issues with *overplotting* for your plot, or not, adjust the position of the points so that they *vary freely along both the x and y axes*. Feel free to modify other aesthetics to address overplotting as well.
7. Save your plot script file.

Make sure that your script file is saved in the appropriate project directory.

Part 2: Creating a Point Plot: Overplotting with Constraints

Using the same data set, create a new version of your plot.

1. Add a new code section to your script that recreates the point plot, now adjusting the position of the points so that they *do not vary freely along either the x or y axis*. Feel free to modify alpha to address overplotting as well if you plot would benefit from it.
2. Re-save your plot script file.

Part 3: Creating Bar Plots: Stacked and Grouped

Stacked and grouped *bar* plots or *column* plots serve different purposes. Even more, stacked plots can represent raw values or they can be conditionalize to represent proportions or percentages of a data set depending on how you stack. Each plot type serves a specific purpose and will influence the user's ability to extract meaningful information more or less effortfully. You will create both stacked and grouped visualizations.

1. Add a new code section to your script where you will create a stacked bar plot to understand your data. **Ideally, you would create a separate script for this plot in order to compartmentalize code to eliminate issues.**
2. Add a new code section to your script that creates a grouped bar plot to understand your data.
3. Re-save your plotting script file.
4. Each data visualization has a goal and should be chosen intentionally to facilitate your story telling. Consider what information you might wish to communicate with the two plot types. Consider how you will facilitate the user experience by using one plot type instead of, or in conjunction with, another plot. Keep a record of your thoughts to inform you future plots.
5. Consider how you would walk your audience through each visualization. With which plot element will you start? What will you say? What are you communicating? What is the story? For in-class presentations, do not assume that your audience is skilled at examining and interpreting data visualizations. Instead, you will need to introduce the plot from the ground up, starting with variables and axes before describing any patterns.

Bonus Practice: Creating Grobs

In some instances, you might wish to combine visualizations together. If so, one way to do this is to create Grobs.

1. Take two of your plots and combine them into a **Grob** with 1 column.
2. Take two of your plots and combine them into a **Grob** with 2 columns.